



HSB

Hochschule Bremen
City University of Applied Sciences

HOCHSCHULE BREMEN
FAKULTÄT 4 - ELEKTROTECHNIK UND INFORMATIK
INTERNATIONALER STUDIENGANG MEDIENINFORMATIK (B.Sc.)

EXPOSÉ

Nachhaltige Persistenz in Webanwendungen

**– Eine Analyse und Implementation verschiedener
Methoden von Datenspeicherung anhand eines
Planspiels –**

Marvin Uwe Marken (*373116*)

21. Mai 2025 (Version 1.00)

1 Einleitung

Im Rahmen eines Wettbewerbs zum Thema “Diversität in Unternehmen” wurde 2023 ein Planspiel entwickelt. Dort noch in Form eines physischen Kartenspiels, begann man im Anschluss zum Wettbewerb mit der Entwicklung einer digitalen Variante in Form einer Webanwendung. Die Anwendung ist mittlerweile auf einem vollwertig durchführbaren Stand. Bisher ist es aber nur allein aus der Sicht des Spielleiters spielbar. In dieser Arbeit soll, die derzeit noch vereinfachte persistente Speicherung der Sitzungsdaten überarbeitet und für eine Mehrspielerfunktion vorbereitet werden. Dabei wird untersucht, wie die Persistenz möglichst nachhaltig gestaltet werden kann.

2 Problemstellung und Lösungsansatz

2.1 Problemstellung

In der Entwicklung moderner Webanwendungen ist die Suche nach einer geeigneten Methodik zur effizienten Speicherung von Nutzerdaten immer wieder eine Herausforderung. Die Wahl der Persistenz-Methode beeinflusst maßgeblich die Performance, Skalierbarkeit und Benutzererfahrung der Anwendung, aber auch dessen Nachhaltigkeit. Dies kommt durch die geplante Mehrspielerfunktion umso mehr zum Tragen, da Live-Updates der Daten durchgeführt werden müssen.

Ein Mehrspielerspiel erfordert eine Datenhaltung, die schnelle Lese- und Schreibzugriffe ermöglicht, um Echtzeitinteraktionen zwischen Spielern zu unterstützen. Gleichzeitig müssen Persistenz-Lösungen sicherstellen, dass Spielstände zuverlässig gespeichert und bei Unterbrechungen korrekt wiederhergestellt werden können. Herkömmliche Methoden, die nur darauf ausgelegt sind, die Interaktionen eines einzelnen Nutzers zu speichern, reichen hier nicht aus. Um eine gute Nachhaltigkeit zu erreichen, sollten zudem auch große Datenverarbeitungen vermieden werden, wie z.B. die gesamte Spielsitzung bei allen Spielern nach jeder Interaktion zu aktualisieren oder Daten(-strukturen) zu wiederholen, die vermieden werden können. Folgende Leitfragen (ab hier als *LF* abgekürzt) werden hierbei gestellt und im Laufe der Arbeit behandelt:

- LF1** Wie beeinflusst die Wahl der Persistenz-Technologie (relationale gegenüber nicht-relationalen Datenbanken) die Skalierbarkeit des Planspiels?
- LF2** Welche Technologie bietet die beste Performance bzgl. Lese-/Schreibgeschwindigkeit und Fehleranfälligkeit für die Persistenz von Echtzeitdaten für eine spätere Mehrspielerfunktion?
- LF3** Wie kann die Persistenz in dem Planspiel ressourcenschonend und energieeffizient gestaltet werden?

2.2 Lösungsansatz

Bisher wurde die Spielsitzung aus einer JSON¹-Datei geladen, Änderungen im LocalStorage gespeichert und aus diesem bei Neuverbindung wieder geladen. Diese Art der Persistenz bietet keine Möglichkeit einer Mehrspielerfunktion, da Änderungen nur im Speicher desjenigen Nutzers vorhanden sind. Daher sollen die Daten auf dem Server gespeichert und über eine API bereitgestellt werden.

¹JavaScript Object Notation

2.2.1 **LF1:** Skalierbare Technologie

2.2.2 **LF2:** Performante Technologie

2.2.3 **LF3:** Nachhaltige Technologie

3 Literaturübersicht

- **Angular 16 Dokumentation**[1] In dieser Dokumentation wird das vom Planspiel genutzte Framework Angular 16 erklärt.

4 Tools

Hardware:

- Notebook - Lenovo ThinkPad P15v Gen 1
- CPU - Intel Core i7-10875H
- RAM - 64GB DDR4
- Server - (Bei den HEC-Admins anfragen?)

Implementationstools:

- JetBrains WebStorm Entwicklungsumgebung
- Google Chrome DevTools

Software-Bibliotheken:

- Angular
- SQLite (?)
- MongoDB (?)
- Docker

Versionsverwaltung:

- Git - Versionskontrollsoftware

5 Vorläufige Gliederung

Nachfolgend die vorläufige Gliederung der Thesis. Es gilt zu beachten, dass vor allem die prototypische Realisierung sich radikal von der Planung unterscheiden kann.

Eigenständigkeitserklärung

Abstract

1. Einleitung

1.1. Problemfeld

1.2. Ziele der Arbeit

1.3. Lösungsansatz

1.4. Aufbau der Arbeit

2. Grundlagen

2.1. Angular

2.2. Datenbanken

2.2.1. Relationale Datenbanken

2.2.2. Nicht-Relationale Datenbanken

2.3. Docker

3. Verwandte Arbeiten

4. Konzeption

5. Prototypische Realisierung

5.1. Wahl der Realisierungsplattform

5.2. Festlegung des Realisierungsumfangs

5.3. Ausgewählte Realisierungsaspekte

5.4. Qualitätssicherung

5.5. Zusammenfassung

6. Evaluation

6.1. Überprüfung funktionaler Anforderungen

6.2. Überprüfung nicht-funktionaler Anforderungen

7. Zusammenfassung und Ausblick

7.1. Zusammenfassung

7.2. Ausblick

Literaturverzeichnis

6 Zeitplanung

Geplanter Starttermin: 31.03.2025

Bearbeitungsdauer: 9 Wochen

Tabelle 1 stellt die geplanten Arbeitspakete und Meilensteine dar:

Tabelle 1: Arbeitspakete und Meilensteine

M1	Offizieller Beginn der Arbeit	31.03.2025
	• Verfassen von Kapitel 1 (Einleitung)	1 Woche
M2	Recherche & Grundlagen	07.04.2025
	• Recherche • Verfassen von Kapitel 2 (Grundlagen) • Verfassen von Kapitel 3 (Verwandte Arbeiten)	2 Wochen
M3	Konzeption	21.04.2025
	• Konzeption • Verfassen von Kapitel 4 (Konzeption)	2 Wochen
M4	Implementation	05.05.2025
	• Implementation • Verfassen von Kapitel 5 (Realisierung)	2 Wochen
M5	Evaluation	19.05.2025
	• Verfassen von Kapitel 6 (Evaluation) • Verfassen von Kapitel 7 (Zusammenfassung & Ausblick)	1 Woche
M6	Erste Fassung vollständige Thesis	26.05.2025
	• Korrekturlesen • Drucken & binden lassen	1 Woche
M7	Abgabe der Thesis	02.06.2025

Literatur

Online-Quellen

- [1] Ohne angegebenen Verfasser. *Angular - Introduction to the Angular docs*. Google. URL: <https://v16.angular.io/docs> (besucht am 26.03.2025).